

Relatório de integração com MOM — Sprint 2

Objetivo

Integrar um middleware orientado a mensagens (MOM) ao backend do FestaLink e usar essa camada para tornar a notificação do prestador assíncrona, atendendo ao item 2.2 do enunciado, que exige arquitetura orientada a eventos e veda polling contínuo no app do prestador.

Tecnologias escolhidas

A escolha foi Redis Pub/Sub provisionado via Docker Compose, em container dedicado ao projeto (`festalink-redis`, porta 6380 do host). O modo pub/sub do Redis casa com a topologia 1 produtor : 1 consumidor que o caso exige e dispensa configuração de filas, exchanges ou ACKs. O cliente `ioredis` foi adotado no lado Node por permitir manter duas conexões separadas — uma em modo normal para publicar e outra em modo subscriber — sem reinventar o protocolo.

Para empurrar o evento ao app do prestador foi adotada a biblioteca `ws`, que oferece um WebSocket server mínimo, sem protocolo proprietário em cima. A autenticação no handshake reaproveita o `TokenService` já usado pelo middleware HTTP, mantendo uma única fonte de verdade para JWT.

Arquitetura final

Três adapters foram introduzidos na camada externa, todos implementando portas que vivem na camada de Use Cases:

- `RedisEventPublisher` publica envelopes JSON no canal `festalink:eventos`.
- `RedisEventConsumer` assina o mesmo canal e despacha cada mensagem para uma função de callback configurada no composition root.
- `WebSocketNotifier` mantém um pool de conexões por `prestadorId` e expõe `notificarPrestadorDoSalao`, que consulta o repositório de salões e empurra o evento ao dono.

Os use cases `CriarReserva`, `AprovarReserva` e `RecusarReserva` ganharam uma dependência opcional `eventPublisher`. Quando presente, eles publicam o evento de domínio correspondente em microtarefa, depois do commit da transação. Quando ausente — útil em testes e em execução sem Redis —, os use cases continuam funcionando normalmente, sem efeitos colaterais.

O wiring acontece em `composition.js` e respeita a regra de dependência: domínio e use cases não conhecem `ioredis` nem `ws`, só interfaces. Trocar o pub/sub do Redis por outro broker ou separar o consumer em outro processo é uma mudança contida no adapter, sem alterar nada do domínio.

Demonstração da comunicação assíncrona

Foi escrito um cliente WebSocket de teste em `backend/scripts/ws-client.js`. Ele conecta no servidor passando um JWT de prestador na query string e imprime no console todo evento recebido.

No teste manual, com o backend rodando e um prestador autenticado pelo cliente WS, foi disparado o fluxo:

1. Cliente envia `POST /reservas` via REST.
2. Backend persiste, publica `ReservaSolicitada` no canal Redis.
3. O consumer dentro do próprio backend lê o evento e o repassa ao notificador.
4. O socket aberto pelo prestador recebe o envelope.
5. Em seguida, o prestador chama `PATCH /reservas/:id/aprovar`. O ciclo se repete com `ReservaAprovada`.

Os logs do servidor mostram cada passagem pelo consumer (`[consumer] ReservaSolicitada salario=4 reserva=4`), e o cliente WS registra a chegada dos dois eventos com timestamps menores que 50 ms de diferença entre a publicação e a entrega.

Suite de testes automatizados

A cobertura Jest passou de 10 para 11 suites e de 127 para 133 testes. Os testes novos verificam que `CriarReserva`, `AprovarReserva` e `RecusarReserva` chamam `eventPublisher.publicar` exatamente uma vez no caminho feliz, com o `type` e o `payload` corretos, e que não publicam nada quando a operação principal falha por autorização ou recurso inexistente. A regressão da Sprint 1 continua passando integralmente.

Limitações conhecidas e próximos passos

A publicação não é transacionalmente consistente com o INSERT — se o processo morrer entre o commit e o `PUBLISH`, o evento é perdido. Para a Sprint 2 isso é aceitável, dado o escopo acadêmico e a simplicidade do pub/sub. Uma versão de produção usaria padrão Outbox: persistir o evento na mesma transação e ter um worker enviando ao broker em segundo plano.

O WebSocket atual mantém o pool em memória do processo único do backend. Em deploy escalado horizontalmente, dois sockets para o mesmo prestador podem cair em processos diferentes; nesse caso, o `PUBLISH` continua chegando a todos os processos via Redis, mas o `notificarPrestadorDoSalao` só dispara onde o socket está aberto. A fan-out é correta dentro do desenho atual e suporta escala leve.

A Sprint 3 adicionará o app cliente Flutter consumindo a REST com polling, conforme o item 3.4 do enunciado. A Sprint 4 adicionará o app prestador em Flutter que abrirá o WebSocket descrito aqui, completando o fluxo ponta-a-ponta.